

Programme principal

Javions – étape 11

1. Introduction

Le but de cette dernière étape est de terminer le projet en écrivant les parties manquantes de l'interface graphique, ainsi que le programme principal.

2. Concepts

2.1. Interface finale

L'interface finale de Javions est visible ci-dessous. De haut en bas, elle est composée de trois parties :

- 1. la vue des aéronefs, avec la carte en arrière-plan,
- 2. la ligne d'état, qui montre le nombre d'aéronefs visibles et de messages reçus,
- 3. la table des aéronefs.

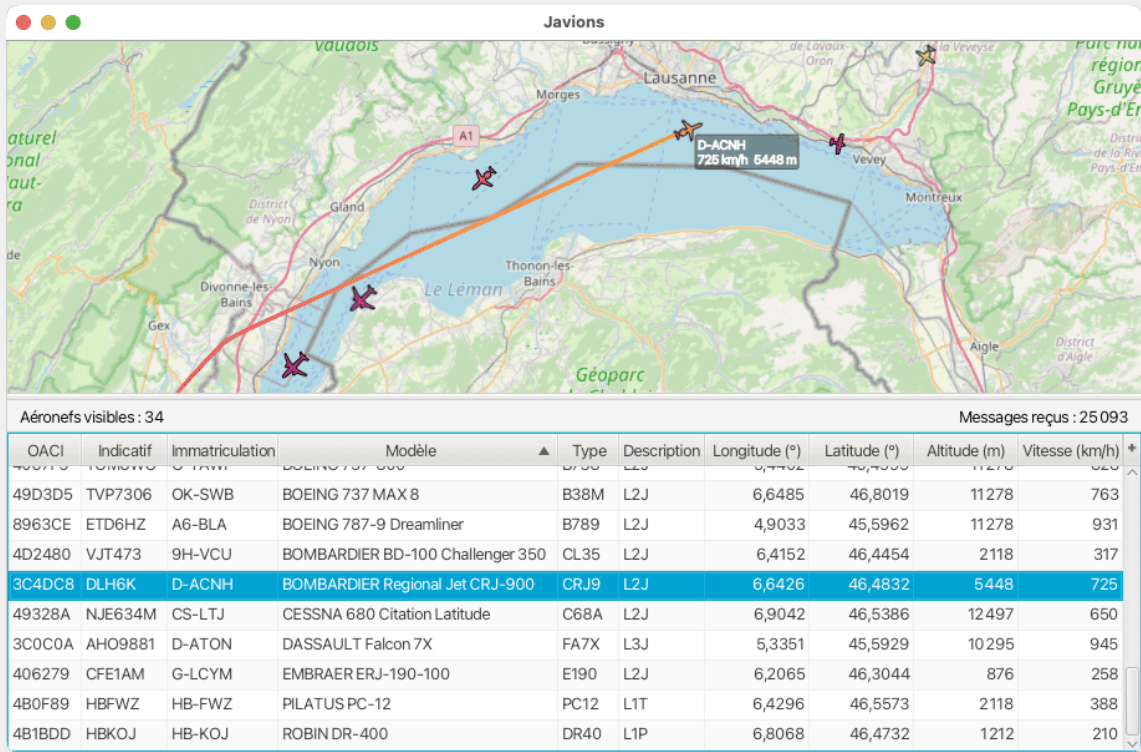


Figure 1 : Interface graphique finale de Javions

Le nombre d'aéronefs « visibles » donné dans la ligne d'état est simplement le nombre de lignes dans la table, et ne tient pas compte du fait que ces aéronefs apparaissent ou non sur la carte. Le nombre de messages reçus, quant à lui, est celui de la totalité des messages considérés comme valides par notre programme depuis le début de son exécution.

La sélection d'un aéronef dans la table provoque sa sélection dans la vue des aéronefs, et inversement. Un clic double dans la table recentre la vue des aéronefs sur celui actuellement sélectionné – s'il y en a un.

2.2. Réception des messages ADS-B

Javions est principalement destiné à être utilisé avec une radio AirSpy, dont les échantillons sont démodulés pour obtenir les messages ADS-B provenant des aéronefs.

Pour cette raison, lorsqu'on l'exécute sans lui passer d'arguments, Javions s'attend à recevoir des échantillons au format utilisé par cette radio depuis ce que l'on nomme son **entrée standard** (*standard input*). Comme nous le verrons plus bas, l'entrée standard d'un programme est représentée en Java par le flot `System.in`.

Toutefois, pour effectuer des tests, il est intéressant de pouvoir utiliser des messages ADS-B enregistrés précédemment dans un fichier. Il est donc aussi possible d'exécuter Javions en lui passant le nom d'un fichier dont le format est identique à celui fourni à [l'étape 7](#) (`messages_20230318_0915.bin`). Dans ce cas, l'horodatage des messages est respecté, c.-à-d. qu'un message n'est traité que lorsqu'une durée égale à son horodatage s'est écoulée depuis le début de l'exécution du programme. Cela garantit que les avions se déplacent à vitesse réelle sur la carte.

Indépendamment du mode dans lequel il fonctionne, Javions utilise ce que l'on nomme un fil d'exécution séparé pour lire les messages, concept qui n'a pas été vu au cours et est donc décrit ci-après.

2.3. Fils d'exécution

Un **fil d'exécution** (*thread of execution*, ou simplement *thread*) est une exécution indépendante d'un programme.

Pour comprendre intuitivement cette idée assez abstraite, on peut imaginer une personne désirant préparer un repas constitué d'une entrée et d'un plat principal. Si cette personne est seule, elle n'a pas d'autre choix que de réaliser chacun des deux plats – l'entrée et le plat principal – de manière séquentielle, c.-à-d. l'un après l'autre. Par contre, si une autre personne propose de l'aider, alors elles peuvent se partager les tâches : l'une peut réaliser l'entrée pendant que l'autre réalise le plat principal. Ces deux personnes travaillent alors de manière indépendante, en parallèle, et dès que la plus lente des deux a terminé le plat dont elle a la charge, le repas est prêt.

Un programme est similaire à une recette. Jusqu'à présent, les programmes Java que vous avez écrits ont toujours été exécutés de manière séquentielle, comme si une seule « personne » s'en chargeait. Cette « personne » correspond justement à un fil d'exécution. Et exactement comme dans l'exemple ci-dessus, il est possible de faire en sorte qu'un seul et même programme soit exécuté par plusieurs fils d'exécution travaillant en parallèle, généralement en se chargeant chacun d'une partie différente du programme.

Lorsqu'on utilise ainsi plusieurs fils d'exécution, on fait ce que l'on nomme de la **programmation concurrente** (*concurrent programming*). Il s'agit d'un sujet bien trop vaste pour être couvert dans le cadre de ce cours, et vous l'étudierez en détail dans la suite de vos études. Seules les quelques bases nécessaires à la réalisation du projet sont donc expliquées ci-dessous.

2.3.1. Fils d'exécution en Java

Tous les programmes Java que vous avez écrits jusqu'ici n'étaient constitués que d'un seul fil d'exécution¹, généralement nommé le **fil principal** (*main thread*). Il est toutefois possible de créer plusieurs fils indépendants au sein d'un seul et même programme, comme nous allons l'illustrer.

Pour ce faire, nous utiliserons un programme d'exemple, dont la première version est tout à fait classique et ne comporte qu'un seul fil d'exécution :

```
public final class ThreadsExample {
    public static void main(String[] args) {
        print26('a');
        print26('A');
        System.out.print('_');
    }

    private static void print26(char c) {
        for (int i = 0; i < 26; i++)
            System.out.print((char)(c + i));
    }
}
```

Lorsqu'on exécute ce programme, il affiche bien entendu la ligne ci-dessous, le premier appel à `print26` affichant l'alphabet en minuscule, le second affichant l'alphabet en majuscule, et le dernier appel à `print` affichant un caractère souligné (_):

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ_
```

On peut maintenant modifier ce programme pour faire en sorte que chacun des appels à `print26` soit fait par un fil d'exécution séparé, représenté par une instance de [Thread](#). La lambda passée au constructeur de cette classe contient le code que le fil exécutera dès qu'il aura été démarré, ce qui se fait en appelant sa méthode [start](#).

```
public final class ThreadsExample {
    public static void main(String[] args) {
        new Thread(() -> print26('a')).start();
        new Thread(() -> print26('A')).start();
        System.out.print('_');
    }
}
```

```

    }
    // ... print26 identique à avant
}

```

Lorsqu'on exécute cette version modifiée du programme, trois fils d'exécution fonctionnent en parallèle : le fil principal, qui crée et démarre chacun des deux fils auxiliaires puis affiche un caractère souligné ; le premier fil auxiliaire, qui affiche l'alphabet en minuscule ; et le second fil auxiliaire qui affiche l'alphabet en majuscule.

Étant donné que ces trois fils s'exécutent en parallèle, il est possible que l'alphabet minuscule, l'alphabet majuscule et le caractère souligné soient entrelacés de manière arbitraire, plutôt que d'être affichés dans cet ordre comme initialement. On peut observer cela en exécutant plusieurs fois de suite le programme ci-dessus. Ainsi les dix lignes suivantes ont été obtenues en le lançant dix fois de suite :

```

abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ_
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ_
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ_
abcde_ABCDEFGHIJKLMNOPQRSTUVWXYZfghijklmnopqrstuvwxyz
abcdefghijklmnopqrstuvwxyz_ABCDEFGHIJKLMNOPQRSTUVWXYZ
abc_dABCDEFGHIJKLMNOPQRSTUVWXYZefghijklmnopqrstuvwxyz
abcdeABCDEFGHIJKLMNOPQRSTUVWXYZ_fghijklmnopqrstuvwxyz
abcdeABCDEFGHIJKLMNOPQRSTUVWXYZ_fghijklmnopqrstuvwxyz
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ_
abcdeAB_CDEFGHIJKLMNOPQRfghijklmnopqrstuvwxyzSTUVWXYZ

```

Comme on le voit, différentes exécutions peuvent donner des résultats différents, car chaque fil procède de manière indépendante, à son propre rythme. Il est toutefois important de comprendre que chacun des fils s'exécute de manière séquentielle, comme d'habitude. Donc les lettres des alphabets minuscule et majuscule apparaissent *toujours* dans le bon ordre. Ce qui varie est l'entrelacement des différents alphabets et du caractère souligné.

2.3.2. Fils d'exécution et JavaFX

Au démarrage d'une application JavaFX, un fil d'exécution nommé le **fil (d'application) JavaFX** (*JavaFX [Application] Thread*) est automatiquement créé par la méthode `launch`. C'est ce fil qui se charge ensuite d'appeler la méthode `start`.

Le fil JavaFX a pour but de gérer l'interface graphique, et c'est donc lui qui exécute, par exemple, tous les gestionnaires d'événement attachés au graphe de scène. Il faut de plus impérativement que ce soit lui qui exécute la totalité du code modifiant l'interface graphique. Il est très important de se souvenir de cela, car le non-respect de cette règle produit des plantages difficiles à diagnostiquer. De plus, il faut aussi être conscient du fait que tout blocage du fil JavaFX provoque un blocage de l'interface graphique. Il ne

faut donc jamais bloquer ce fil pour une durée indéterminée.

2.3.3. Fils d'exécution Javions

Dans le cadre de Javions, nous utiliserons deux fils d'exécution. Le premier d'entre eux sera simplement le fil JavaFX, dont le but est de gérer l'interface graphique, comme nous l'avons dit. Le second fil sera quant à lui chargé d'obtenir les messages provenant des aéronefs, soit en démodulant le signal radio, soit en lisant les messages depuis un fichier.

Pour communiquer entre eux, ces deux fils utiliseront une file (*queue*) de message partagée. Le fil chargé d'obtenir les messages des aéronefs y placera les messages reçus, tandis que le fil JavaFX les en extraira périodiquement afin de mettre à jour l'interface graphique.

Il faut toutefois faire attention au type de file utilisé pour communiquer, car celles que nous avons vues jusqu'à présent – p. ex. `ArrayDeque` – ne peuvent **pas** être partagées entre plusieurs fils d'exécution. On dit qu'elles ne sont pas *thread safe*. En effet, si un fil d'exécution ajoute, par exemple, un élément à une file de type `ArrayDeque` alors qu'un autre fil d'exécution en retire simultanément un élément, il est possible que la file soit corrompue en conséquence. Là aussi, les problèmes que cela produit sont difficiles à diagnostiquer.

Dès lors, chaque fois que l'on désire ainsi partager des collections – ou n'importe quel autre valeur non immuable – entre plusieurs fils d'exécution, il faut impérativement s'assurer que ces objets partagés sont *thread safe*.

La bibliothèque Java offre plusieurs collections non immuables et *thread safe* – donc prévues pour être partagées par plusieurs fils d'exécution – dans le paquetage `java.util.concurrent`. Dans le cadre de ce projet, nous utiliserons uniquement `ConcurrentLinkedQueue`, qui implémente l'interface `Queue` et représente donc une file.

3. Mise en œuvre Java

3.1. Correction d'erreurs

Avant d'entamer cette étape, il convient de corriger certaines erreurs qui ont été faites durant les étapes précédentes.

3.1.1. Gestion de la trajectoire

La §3.2.1.3 de [l'étape 7](#) (*Calcul de la trajectoire*) suggère une technique de calcul de la trajectoire qui est inutilement compliquée. Il est possible de la simplifier ainsi :

- lorsqu'une nouvelle position est reçue, si l'altitude est connue, alors la paire (position courante, altitude courante) est ajoutée à la trajectoire,

- lorsqu'une nouvelle altitude est reçue, si la position est connue :
 - si la trajectoire est vide, alors la paire (position courante, altitude courante) est ajoutée à la trajectoire,
 - si l'horodatage du message courant est identique à celui du message qui a provoqué le dernier ajout à la trajectoire, alors son dernier élément est remplacé par la paire (position courante, altitude courante).

3.1.2. Purge des avions

L'étape 7 spécifie que la méthode purge de la classe `AircraftStateManager` doit supprimer de l'ensemble des états observables ceux correspondants à des aéronefs dont aucun message n'a été reçu depuis une minute. Il n'est toutefois pas précisé si les accumulateurs correspondant à ces états doivent également être supprimés de la table.

Même si conserver les accumulateurs pourrait avoir certains avantages – en particulier si un aéronef est « perdu » durant plus d'une minute puis réapparaît –, cela présente également un inconvénient majeur : la table ne fait que grandir, potentiellement jusqu'au point où elle consomme la totalité de la mémoire à disposition et provoque le plantage du programme. J'avions étant destiné à être exécuté sur une longue période, cela n'est pas souhaitable.

Dès lors, la méthode purge de `AircraftStateManager` doit supprimer aussi bien les accumulateurs que les états des aéronefs dont aucun message n'a été reçu dans la minute précédant la réception du dernier message passé à `updateWithMessage`.

3.1.3. Classe de style pour les colonnes de la table

La version originale de l'étape 10 spécifiait que la classe de style `numeric` devait être attachée aux cellules individuelles des colonnes numériques. Or il se trouve que l'attacher aux colonnes elles-mêmes fonctionne également, et comme cela est nettement plus simple, c'est ce qu'il faut faire.

3.2. Classe `StatusLineController`

La classe `StatusLineController` du sous-paquetage `gui`, publique et finale, gère la ligne d'état. Elle possède un constructeur par défaut, qui construit le graphe de scène, ainsi que trois méthodes publiques, qui sont :

- une méthode nommée p. ex. `pane`, qui retourne le panneau contenant la ligne d'état,
- une méthode nommée p. ex. `aircraftCountProperty`, qui retourne la propriété (modifiable) contenant le nombre d'aéronefs actuellement visibles,
- une méthode nommée p. ex. `messageCountProperty`, qui retourne la propriété (modifiable) contenant le nombre de messages reçus depuis le début de l'exécution du programme.

La troisième de ces propriétés doit contenir une valeur de type `long`, et pas `int`, car il n'est pas totalement impossible qu'en laissant tourner le programme longtemps, le nombre de messages reçus dépasse `Integer.MAX_VALUE`².

3.2.1. Graphe de scène

Le graphe de scène de la ligne d'état est extrêmement simple et consiste en un panneau de type `BorderPane` auquel la feuille de style `status.css` est attachée et dont :

- la zone gauche (*left*) contient une instance de `Text` indiquant le nombre d'aéronefs visibles,
- la zone droite (*right*) contient une instance de `Text` indiquant le nombre de messages reçus depuis le début de l'exécution du programme,
- les trois autres zones (*top*, *bottom* et *center*) sont vides.

3.2.2. Auditeurs et liens

La propriété `textProperty` de l'instance de `Text` placée dans la zone gauche doit être liée de manière à ce que son contenu soit toujours égal à la chaîne ci-dessous :

Aéronefs visibles : ???

dans laquelle les trois points d'interrogation représentent le contenu de la propriété contenant le nombre d'aéronefs actuellement visibles.

De même, la propriété `textProperty` de l'instance de `Text` placée dans la zone droite doit être liée de manière à ce que son contenu soit toujours égal à la chaîne :

Messages reçus : ???

dans laquelle les trois points d'interrogation représentent le contenu de la propriété contenant le nombre de messages reçus depuis le début de l'exécution du programme.

3.3. Classe `Main`

La classe `Main` du sous-paquetage `gui`, publique et finale, contient le programme principal. Comme toute classe représentant une application JavaFX, elle hérite d'`Application`, et est dotée d'une méthode `main` qui ne fait rien d'autre qu'appeler `launch`.

La mise en œuvre de la méthode `start` démarre l'application en construisant le graphe de scène correspondant à l'interface graphique, démarrant le fil d'exécution chargé d'obtenir les messages, et enfin démarrant le « minuteur d'animation » chargé de mettre à jour les états d'aéronefs en fonction des messages reçus. Le minuteur se charge également de purger, une fois par seconde, les aéronefs dont aucun message n'a été reçu

depuis une minute.

Le fil d'exécution chargé d'obtenir les messages les obtient soit d'un démodulateur ADS-B connecté à l'entrée standard ([System.in](#)) – lorsque le programme est lancé sans arguments – soit depuis un fichier dont le nom est le premier argument passé au programme sinon. Afin qu'il n'empêche pas le programme de se terminer lorsque l'utilisateur ferme la fenêtre, ce fil doit être un « *daemon* » – c.-à-d. qu'il faut appeler sa méthode [setDaemon](#) avec `true` en argument.

La méthode `start` doit être écrite de manière à ce que :

- la base de données soit lue depuis la ressource `/aircraft.zip`,
- le niveau de zoom initial soit 8, et le coin haut-gauche de la région affichée dans la fenêtre ait les coordonnées Web Mercator $x = 33\,530$, $y = 23\,070$,
- les tuiles soient obtenues depuis le serveur nommé `tile.openstreetmap.org`,
- le cache disque soit stocké dans le sous-dossier nommé `tile-cache` du dossier courant.

De plus amples détails à ce sujet sont donnés dans les conseils de programmation plus bas.

3.3.1. Graphe de scène

Le graphe de scène de l'application consiste en un panneau de type [SplitPane](#), d'orientation verticale, possédant deux fils qui sont, dans l'ordre :

- un panneau de type [StackPane](#) superposant la vue des aéronefs au fond de carte,
- un panneau de type [BorderPane](#) dont la zone centrale (*center*) est occupée par la table des aéronefs et la zone supérieure (*top*) par la ligne d'état.

Le titre de la fenêtre principale de l'application – la valeur de type `Stage` passée à `start` – est `Javions`, et ses dimensions minimales – réglables au moyen des méthodes [setMinWidth](#) et [setMinHeight](#) – sont de 800×600 unités.

3.3.2. Liens et auditeurs

La propriété de la ligne d'état contenant le nombre d'aéronefs visibles doit être liée à la taille de l'ensemble des aéronefs exporté par le gestionnaire d'états d'aéronefs (de type `AircraftStateManager`). Cela peut se faire facilement au moyen de la méthode [size](#) de `Bindings`.

La propriété de la ligne d'état contenant le nombre de messages reçus n'est quant à elle liée à rien, mais son contenu est modifié directement par le code chargé d'extraire les messages de la file.

3.3.3. Conseils de programmation

1. Obtention de la base de données

Le code ci-dessous illustre la manière correcte d'obtenir le nom de fichier à passer au constructeur de `AircraftDatabase` à partir du nom de ressource (`/aircraft.zip`).

```
URL u = getClass().getResource("/aircraft.zip");
assert u != null;
Path p = Path.of(u.toURI());
AircraftDatabase db = new AircraftDatabase(p.toString());
```

Ce code est plus simple que celui donné à l'étape 2, et fonctionne même si le chemin d'accès à la base de données comporte des espaces ou caractères spéciaux.

2. Obtention des arguments

Les arguments passés à un programme Java lui sont fournis sous la forme d'un tableau de chaînes de caractères passé à sa méthode `main`.

JavaFX permet d'obtenir une liste de chaînes de caractères contenant ces arguments, en appelant la méthode `getParameters` puis la méthode `getRaw` sur le résultat. Comme expliqué plus haut, si cette liste est vide, alors le programme principal lit les échantillons depuis `System.in`, sinon il lit les messages depuis un fichier dont le nom est le premier élément de la liste d'arguments, les autres étant ignorés.

3. Gestion des messages

Indépendamment de la manière dont les messages sont obtenus – soit par démodulation du signal provenant d'une radio, soit par lecture d'un fichier –, ils le sont par le fil chargé de cette tâche. Ce fil les place dans une file de type `ConcurrentLinkedQueue` d'où ils sont ensuite extraits par le minuteur d'animation décrit ci-après, lui-même exécuté par le fil JavaFX.

Lorsque les messages proviennent de la radio, ils sont placés dans la file dès leur arrivée, mais lorsqu'ils proviennent d'un fichier, il n'y sont placés que lorsqu'une durée égale à leur horodatage s'est écoulée depuis le début de l'exécution du programme.

Les messages sont extraits de la file afin d'être traités par un « minuteur d'animation » de type `AnimationTimer`. Un tel minuteur possède une méthode `handle` qui est appelée périodiquement par le fil JavaFX, à une fréquence non spécifiée – mais généralement supérieure ou égale à 60 Hz. La méthode `handle` du minuteur chargé de traiter les messages vide simplement la file partagée avec le fil chargé d'obtenir les messages, et passe chacun des éléments qu'elle contient à la méthode `updateWithMessage` de l'instance de `AircraftStateManager` chargée de gérer les états des aéronefs. Cela provoque, indirectement, la mise à

jour de l'interface graphique.

Un exemple de définition d'une sous-classe de `AnimationTimer` est donné dans le programme de test de [l'étape 9](#), dont vous pouvez vous inspirer. Faites bien attention à ce que votre méthode `handle` retourne dès que la file des messages est vide, car tant qu'elle s'exécute, l'interface graphique est totalement bloquée.

3.4. Tests

Étant donné que la classe représentant le programme principal doit impérativement avoir le bon nom et posséder une méthode `main` ayant la bonne signature, nous vous fournissons [un fichier de vérification de signature](#) pour cette étape. Il doit être intégré à votre projet comme ceux des étapes 1 à 6. Notez toutefois que les anciens fichiers de vérification de signature ne sont plus nécessaires, et peuvent être supprimés.

Il est important de vérifier que votre programme fonctionne aussi bien lorsqu'on lui fournit – par son entrée standard – des échantillons provenant d'une radio AirSpy, que lorsqu'on lui donne le nom d'un fichier contenant des messages.

Pour vérifier qu'il fonctionne lorsqu'on lui donne le nom d'un fichier contenant des messages, il vous faut définir une « configuration d'exécution » (*run configuration*) dans IntelliJ, en mettant le chemin d'accès au fichier contenant les messages (p. ex. `messages_20230318_0915.bin` de l'étape 7) dans le champ nommé *Program arguments*. En cas de problème, consultez [notre guide à ce sujet](#).

La vidéo ci-dessous montre le comportement du programme lorsqu'on l'exécute en lui passant justement le fichier de messages fourni à l'étape 7. Afin de faciliter sa compréhension, un cercle rouge apparaît brièvement lors d'un clic de la souris, mais ce cercle ne fait bien entendu pas partie de l'interface graphique.

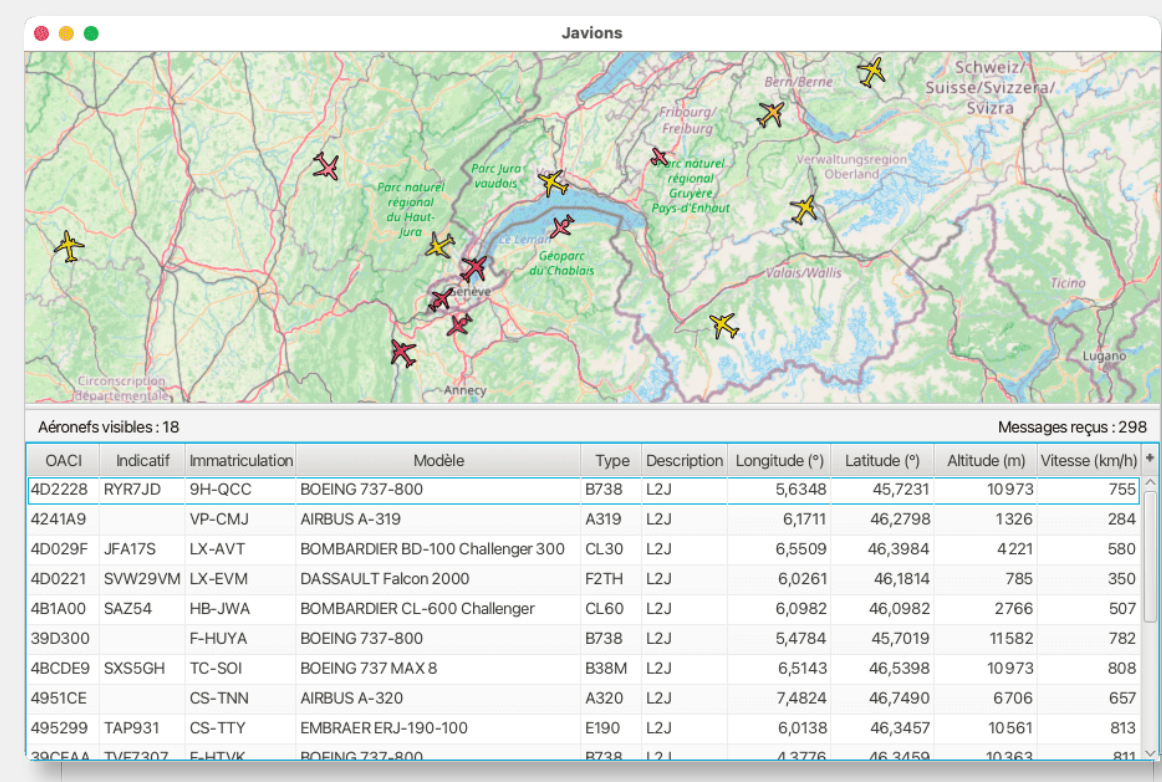


Pour vérifier que votre programme fonctionne également lorsqu'on lui fournit des échantillons via son entrée standard, il vous faut définir une seconde configuration

d'exécution puis demander à IntelliJ de fournir le contenu d'un fichier à l'entrée standard du programme.

Pour cela, créez la configuration d'exécution puis cliquez sur *Modify options* et choisissez *Redirect input*. Ensuite, dans le champ nommé *Redirect input from* nouvellement ajouté, cliquez sur l'icône représentant un dossier puis naviguez jusqu'au fichier contenant les échantillons fourni à l'étape 4 (`samples_20230304_1442.bin`).

En lançant votre programme ainsi, vous devriez constater qu'il démodule correctement les 298 messages contenus dans ces échantillons, met l'interface graphique à jour en fonction, puis plus rien ne se passe. La fenêtre devrait alors avoir l'aspect suivant :



4. Résumé

Pour cette étape, vous devez :

- écrire les classes `StatusLineController` et `Main` selon les indications données ci-dessus,
- tester votre code,
- rendre votre projet dans le cadre du rendu final anticipé, si vous désirez tenter d'obtenir un bonus, ou dans le cadre du rendu final normal sinon ; les instructions concernant ces rendus seront publiées ultérieurement.

N'attendez surtout pas le dernier moment pour effectuer votre rendu, car vous n'êtes pas à l'abri d'imprévus. **Souvenez-vous qu'aucun retard, aussi insignifiant soit-il, ne sera toléré !**

Notes de bas de page

¹ Cela n'est pas tout à fait vrai, entre autres car toute application JavaFX comporte au moins un fil d'exécution en plus du fil principal, qui gère l'interface graphique. Cela dit, vous n'aviez probablement pas conscience de son existence jusqu'à présent.

² Un message ADS-B dure 120 μ s, donc le temps minimum nécessaire à la réception de 2^{31} messages est d'un peu plus de 71 heures. Même si en pratique il faut beaucoup plus de temps pour recevoir autant de messages, il paraît raisonnable d'avoir un peu de marge.